

Módulo 2 Implementación de un modelo de deep learning.

Maxime Vilcocq Parra, A01710550

Problema:

Nunca he sido muy bueno para tocar música. Por ello quise hacer un programa que utilizando inteligencia artificial pudiera generar música solo.

Objetivo:

El objetivo del programa es implementar un algoritmo de deep learning para generar melodías con un estilo predeterminado.

Alcance:

He utilizado 3 sets de datos. 1 set de 101 canciones de música clásica barroca, 1 set de 101 canciones de videojuegos de géneros distintos y 1 set que combina ambos sets previos (202 canciones).

Con esto se generan 3 modelos (1 por cada set de datos).

Con los modelos de cada grupo se generan 5 melodías utilizando estrategias distintas.

Limitaciones:

101 canciones, aunque se pueda escuchar como mucho, puede no ser suficiente para entrenar un modelo que suene bien.

Las canciones de videojuegos seleccionadas son de distintos géneros e incluyen combinaciones de notas muy variadas y complicadas, además la variedad de instrumentos empleados en estas piezas es mucho mayor por lo que podría haber algo de ruido (ya que no separamos, por ejemplo; trompetas de tambores y piano).

Por la naturaleza del problema es difícil manejar un dataset dividido en entrenamiento, pruebas y validación. La evaluación de la calidad de los resultados es subjetiva.

Se puede analizar el parecido con el género a través de espectrogramas, pero esto solo puede hacerse hasta después de generar el modelo y entrenarlo. Por ello, hacer ajustes en el modelo para generar mejoras puede ser bastante costoso.

La música en general tiene es muy variable, esto puede complicar que el modelo aprenda ciertos patrones si no se incluyen en el set de datos.

Algunas estrategias generan mejores resultados que otras dependiendo del modelo y el patrón inicial, (aunque mi elección de nombre sugiera lo contrario) no hay una estrategia mejor que otra. La calidad de la música es subjetiva a quien la escucha.

Problemas enfrentados:

El dataset de videojuegos era demasiado complejo y no era posible que el modelo comprendiera los patrones de este set y del set combinado.

Los modelos generados, especialmente los que contienen el set de datos de videojuegos, suelen repetir varias veces seguidas una sola tecla.

Dificultad para medir la calidad de la música generada por el modelo.

Mejoras:

Mejoras aplicadas:

- Utilizar embeddings para facilitar la comprensión de los patrones.
- Emplear estrategias distintas para generar las melodías.
- Cambiar el número de repeticiones de una tecla permitidas.
- Utilizar distintas longitudes de secuencias para el entrenamiento (utilizo 50, ya que longitudes muy grandes reducen el número de secuencias distintas que se pueden analizar, en melodías cortas a veces no podía generar ninguna).

Posibles mejoras futuras:

- Implementar silencio como una posibilidad.
- Utilizar un set de datos más grande.
- Buscar un modelo pre-entrenado para hacer transfer learning.
- Utilizar un set de datos de un género más homogéneo.
- Probar con más configuraciones del modelo.

- Experimentar más con el número de capas, neuronas y el porcentaje de dropout (bajarlo ya que no nos interesa el overfitting tanto).

Extracción de datos:

Extraje 101 archivos .mid de música clásica barroca de una biblioteca y 101 archivos de música de videojuegos de bitmidi.com.

Entrenamiento de modelo:

El algoritmo que elegí es el de lstm (long short-term memory). Este es un tipo de red neuronal recurrente muy útil para procesar secuencias de datos (como la música). Este tipo de red maneja 3 puertas. Forget gate, input gate y output gate; las cuales sirven para decidir qué partes de la secuencia ya no importan, que notas tienen relevancia y en predecir la siguiente nota respectivamente.

```
from tensorflow.keras.optimizers import Adam
def build_lstm_model(n_vocab, seq_length=50):
    model = Sequential([
        LSTM(256, input_shape=(seq_length, 1), return_sequences=True),
        Dropout(0.3),
        LSTM(256),
        Dense(128, activation='relu'),
        Dropout(0.3),
        Dense(n_vocab, activation='softmax')
    ])
    model.compile(loss='categorical_crossentropy', optimizer=Adam(learning_rate=0.001), metrics=['accuracy'])
    return model
```

La primera capa contiene 256 neuronas que aprenden patrones, recorren la secuencia de notas (50 notas).

Con dropout, se eliminan aleatoriamente el 30% de las conexiones (esto suele ayudar a evitar problemas de overfitting).

Luego hay otra capa LSTM de 256 neuronas que ayuda a resumir los resultados de la primera.

Una capa densa de 128 neuronas que elimina los valores negativos con relu (prepara los valores para la predicción final).

Un nuevo dropout que funciona igual que el anterior.

Y finalmente, una capa densa adicional con una neurona por nota que convierte las predicciones de cada nota a un porcentaje/probabilidad.

Predicciones realizadas:

Con el modelo obtenido se crean 5 melodías distintas empleando distintas estrategias.

Determinista: `generate_notes()` toma la nota más probable y ya. Nos muestra la estructura musical que aprendió el modelo. Puede ser muy repetitiva.

Generación estocástica simple: `generate_notes_creatively()`. Utiliza la probabilidad de las notas para elegir una nota.

Generación creativa con temperatura: `better_generate_notes_creatively()`. Permite introducir un parámetro de “creatividad”. En este programa lo usé principalmente para agregar “riesgo” con los modelos que no eran el clásico que tienden a repetir mucho lo mismo.

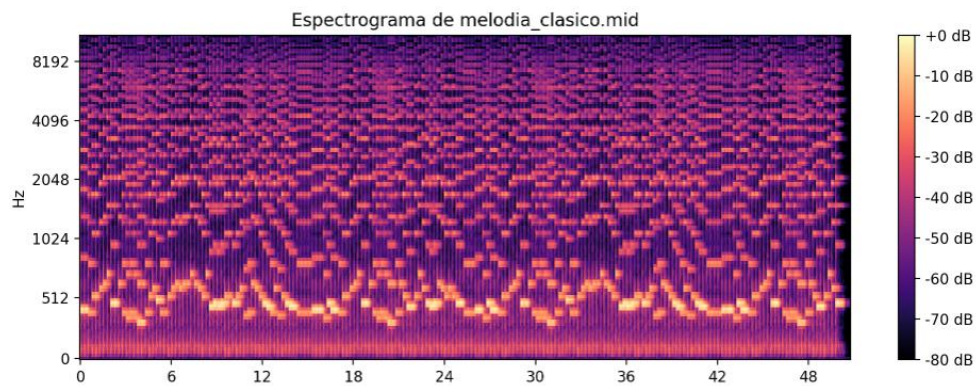
Generación determinista sin repeticiones: `generate_notes_no_repeat()`. Permite controlar que las notas no salgan dos veces seguidas y que no salgan tantas veces en un número determinado de notas (no más de 3 veces en las últimas 7 posiciones).

Generación creativa con temperatura y control de repetición:
`better_generate_notes_creatively_no_repeat()` hace lo mismo que la estrategia anterior agregándole también temperatura.

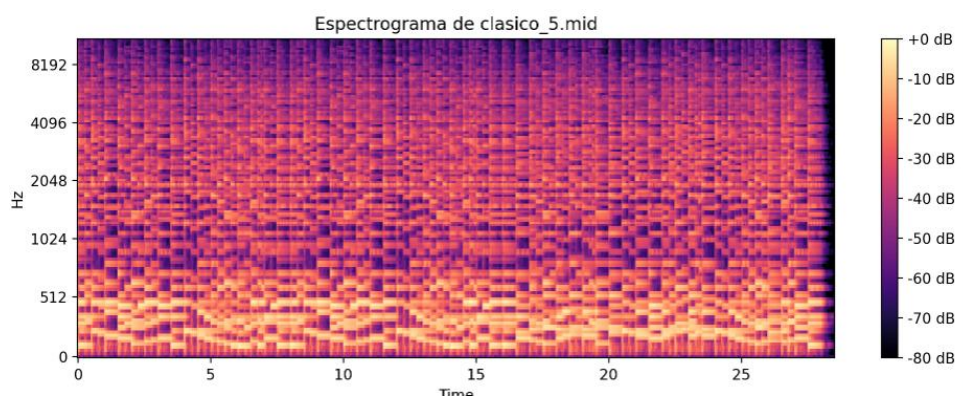
Evaluación de resultados y conclusiones:

Al ser un modelo sencillo, no es posible aprender de forma perfecta el estilo. Sin embargo por los espectrogramas generados, podemos asumir que sí está aprendiendo algo de las canciones del dataset Original.

Para analizar el espectrograma, decidí emplear la melodía creada con la estrategia determinista.

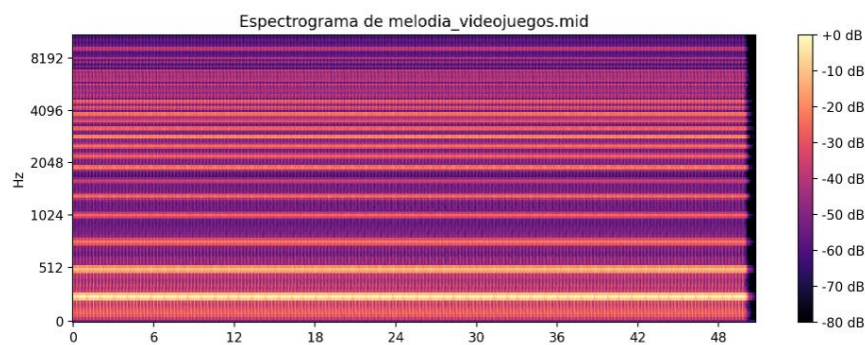


En este espectrograma podemos observar que el modelo aprende un patrón de subidas y bajadas que se puede apreciar mucho en los espectrogramas de música clásica.



A pesar de que las canciones originales, son mucho más complejas, se puede ver claramente esta forma de olas en muchas de las melodías.

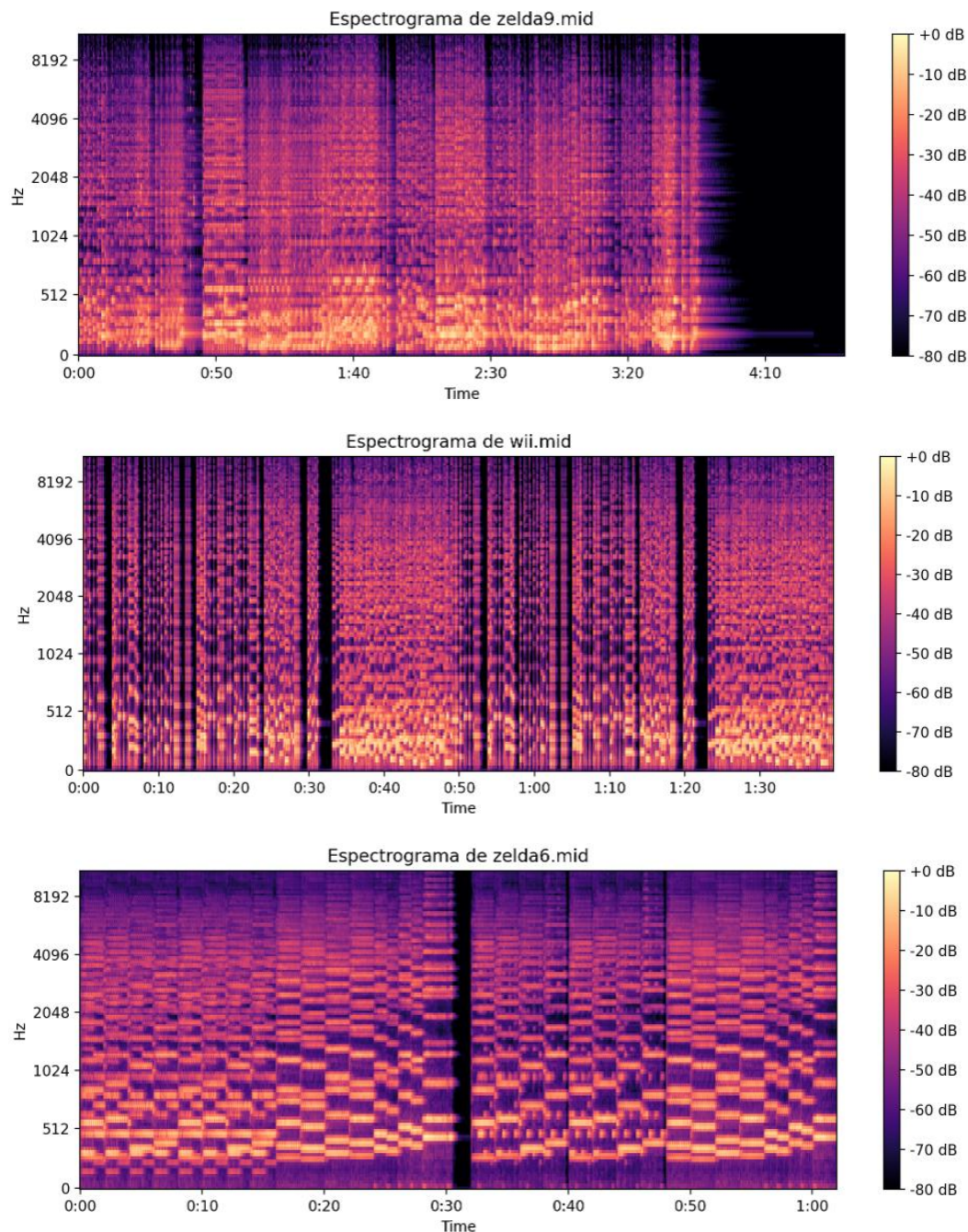
Por otro lado en los videojuegos:



Vemos un resultado no tan bueno (es malo). Se ve la repetición de la misma tecla una y otra y otra vez. Aquí lo que sucede es probablemente de cómo es la música de los

videojuegos que suele repetir mucho las mismas notas. Por ello el modelo aprende que predecir la misma nota una y otra vez es bueno.

Además, aquí quizá haya sido un problema el ruido, la complejidad y la diferencia de estilos de las canciones empleadas.



Aquí podemos ver tres espectrogramas muy distintos, pero que mantienen algo en común, todos tienen muchos segmentos en los que el espectrograma se mantiene

mucho tiempo a la misma altura, lo que probablemente ocasione que la predicción determinista sea tan plana.